Metrics Columns Overview

This document provides a comprehensive list of all prestige, division, OER, pool, grade, and last4 metrics columns and how they are computed.

Prestige Metrics

Prestige metrics track an officer's assignment to high-prestige units over time. These metrics are computed in **CELL 7** of the pipeline.

`prestige_unit`

- **Type**: Binary (0 or 1)
- **Computation**:
  - Binary indicator: `1` if officer is assigned to a prestige unit at this snapshot, `0` otherwise
  - Determined by unit classification logic (uses `df_uic_hierarchy` lookup or 502 output)
- **Source**: Created in CELL 7 based on unit classification

`prestige_sum`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative sum of `prestige_unit` for each officer across all snapshots
  - Formula: `df.groupby('pid_pde')['prestige_unit'].cumsum()`
  - Tracks total number of quarters spent in prestige units up to each snapshot date
- **Source**: Created in CELL 7

`prestige_mean`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Expanding mean of `prestige_unit` for each officer
  - Formula: `df.groupby('pid_pde')['prestige_unit'].transform(lambda x: x.expanding().mean())`
  - Represents the proportion of quarters spent in prestige units up to each snapshot date
- **Source**: Created in CELL 7

`cum_prestige_quarters` (Optional - from OER metrics module)

- **Type**: Integer (cumulative count)

- **Computation**:
  - Cumulative count of quarters in prestige units (same as `prestige_sum`)
  - Computed by `_compute_prestige_metrics()` in `add_cum_oer_metrics_mod_working.py`
  - Formula: `df.groupby('pid_pde')['prestige_unit'].cumsum()`
- **Source**: Optional module in OER metrics computation (Module 6)
- **Note**: Only created if `include_prestige_metrics=True` when calling `add_cumulative_oer_metrics()`

`cum_total_quarters` (Optional - from OER metrics module)

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of total quarters (snapshots) up to each snapshot date
  - Formula: `df.groupby('pid_pde').cumcount() + 1`
- **Source**: Optional module in OER metrics computation (Module 6)

`mean_prestige_quarters` (Optional - from OER metrics module)

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Ratio of prestige quarters to total quarters
  - Formula: `cum_prestige_quarters / cum_total_quarters` (with division by zero handling)
- **Source**: Optional module in OER metrics computation (Module 6)

`prestige_quarters_ratio` (Optional - from OER metrics module)

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Alias for `mean_prestige_quarters` (for consistency with other ratio metrics)
  - Same value as `mean_prestige_quarters`
- **Source**: Optional module in OER metrics computation (Module 6)

Division Metrics

Division metrics track where officers serve and summarize time spent in divisions. These are computed in **CELL 7** using `apply_division_name()` and `add_division_metrics()` from `py_503_hierarchies.py` and are controlled by `DIVISION_CONFIG` in `pipeline_config.py`.

`div_name` (Optional)

- **Type**: String (division name or code)
- **Computation**:
  - Merged from the UIC hierarchy file when `DIVISION_CONFIG['merge_div_name']=True`

- Filled or normalized based on hierarchy lookup
- **Source**: CELL 7

`div_cum_time`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative number of snapshots spent in the current officer's division
  - Formula: `df.groupby('pid_pde')['div_name'].cumcount() + 1` (within division segments)
- **Source**: CELL 7 (`add_division_metrics`)

`div_ratio_time`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Ratio of cumulative time in division to total snapshots
  - Formula: `div_cum_time / total_snapshots` with division by zero handling
- **Source**: CELL 7 (`add_division_metrics`)

`final_div_cpt`

- **Type**: String (division at Captain promotion)
- **Computation**:
  - Captures the division name at the Captain promotion snapshot (if present)
  - Uses the officer's promotion event window to identify the relevant division
- **Source**: CELL 7 (`add_division_metrics`)

`div_<division>_cum_time` (Optional per-division metrics)

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative snapshots spent in each configured division
  - Only created if `DIVISION_CONFIG['create_per_division_metrics']=True`
- **Source**: CELL 7 (`add_division_metrics`)

`div_<division>_ratio_time` (Optional per-division metrics)

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Ratio of snapshots in each division to total snapshots
  - Only created if `DIVISION_CONFIG['create_per_division_metrics']=True`
- **Source**: CELL 7 (`add_division_metrics`)

OER Cumulative Metrics

OER cumulative metrics track an officer's evaluation history (top blocks and evaluations received) up to each snapshot date. These metrics are computed in **CELL 6A** using `add_cumulative_oer_metrics()`.

`cum_tb_rcvd_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of top block (TB) ratings (box score = 50.0) received from **raters** up to each snapshot date
  - Uses backward temporal lookup: finds all OERs with `eval_thru_dt <= snpsht_dt` and `rater_box == 50.0`
  - Each OER is counted exactly once (deduplicated by `eval_id`)
- **Source**: Module 1 in `add_cum_oer_metrics_mod_working.py`

`cum_tb_rcvd_snr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of top block (TB) ratings (box score = 50.0) received from **senior raters** up to each snapshot date
  - Uses backward temporal lookup: finds all OERs with `eval_thru_dt <= snpsht_dt` and `snr_rater_box == 50.0`
  - Each OER is counted exactly once (deduplicated by `eval_id`)
- **Source**: Module 1 in `add_cum_oer_metrics_mod_working.py`

`cum_evals_rcvd_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of evaluations received from **raters** up to each snapshot date
  - Counts any OER with a non-null `rater_box` value (not just top blocks)
  - Uses backward temporal lookup: finds all OERs with `eval_thru_dt <= snpsht_dt` and `rater_box.notna()`
  - Each OER is counted exactly once (deduplicated by `eval_id`)
- **Source**: Module 1 in `add_cum_oer_metrics_mod_working.py`

`cum_evals_rcvd_snr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of evaluations received from **senior raters** up to each snapshot date
  - Counts any OER with a non-null `snr_rater_box` value (not just top blocks)
  - Uses backward temporal lookup: finds all OERs with `eval_thru_dt <= snpsht_dt` and `snr_rater_box.notna()`
  - Each OER is counted exactly once (deduplicated by `eval_id`)
- **Source**: Module 1 in `add_cum_oer_metrics_mod_working.py`

`cum_tb_rcvd_ratio_rtr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Ratio of top blocks to total evaluations from **raters**
  - Formula: `cum_tb_rcvd_rtr / cum_evals_rcvd_rtr` (returns `NaN` if `cum_evals_rcvd_rtr == 0`)
  - Represents the proportion of rater evaluations that were top blocks
- **Source**: Module 1 in `add_cum_oer_metrics_mod_working.py`

`cum_tb_rcvd_ratio_snr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Ratio of top blocks to total evaluations from **senior raters**
  - Formula: `cum_tb_rcvd_snr / cum_evals_rcvd_snr` (returns `NaN` if `cum_evals_rcvd_snr == 0`)
  - Represents the proportion of senior rater evaluations that were top blocks
- **Source**: Module 1 in `add_cum_oer_metrics_mod_working.py`

Grade-Based Metrics

Grade-based metrics track OER performance separately for Company Grade (Captain) and Field Grade (Major) periods. These metrics are computed in **CELL 6A** using `add_cumulative_oer_metrics()` with `include_grade_metrics=True`.

Company Grade (Captain) Metrics

`co_grade_tb_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of top blocks from **raters** during **Captain** grade period
  - Only counts OERs where officer was Captain (`rank_pde == 'CPT'` or `dor_cpt <= eval_thru_dt < dor_maj`)
  - Uses backward temporal lookup with grade filtering
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`co_grade_tb_snr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of top blocks from **senior raters** during **Captain** grade period
  - Only counts OERs where officer was Captain
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`co_grade_evals_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of evaluations from **raters** during **Captain** grade period
  - Only counts OERs where officer was Captain
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`co_grade_evals_snr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of evaluations from **senior raters** during **Captain** grade period
  - Only counts OERs where officer was Captain
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`co_grade_tb_ratio_rtr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Ratio of top blocks to evaluations from **raters** during **Captain** grade period
  - Formula: `co_grade_tb_rtr / co_grade_evals_rtr` (returns `NaN` if denominator is 0)
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`co_grade_tb_ratio_snr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
  - Ratio of top blocks to evaluations from **senior raters** during **Captain** grade period
  - Formula: `co_grade_tb_snr / co_grade_evals_snr` (returns `NaN` if denominator is 0)
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

Field Grade (Major) Metrics

`field_grade_tb_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of top blocks from **raters** during **Major** grade period
  - Only counts OERs where officer was Major (`rank_pde == 'MAJ'` or `dor_maj <= eval_thru_dt`)
  - Uses backward temporal lookup with grade filtering
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`field_grade_tb_snr`

- **Type**: Integer (cumulative count)
- **Computation**:

    - Cumulative count of top blocks from **senior raters** during **Major** grade period
    - Only counts OERs where officer was Major
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`field_grade_evals_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
    - Cumulative count of evaluations from **raters** during **Major** grade period
    - Only counts OERs where officer was Major
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`field_grade_evals_snr`

- **Type**: Integer (cumulative count)
- **Computation**:
    - Cumulative count of evaluations from **senior raters** during **Major** grade period
    - Only counts OERs where officer was Major
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`field_grade_tb_ratio_rtr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
    - Ratio of top blocks to evaluations from **raters** during **Major** grade period
    - Formula: `field_grade_tb_rtr / field_grade_evals_rtr` (returns `NaN` if denominator is 0)
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

`field_grade_tb_ratio_snr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
    - Ratio of top blocks to evaluations from **senior raters** during **Major** grade period
    - Formula: `field_grade_tb_snr / field_grade_evals_snr` (returns `NaN` if denominator is 0)
- **Source**: Module 2 in `add_cum_oer_metrics_mod_working.py`

Last4 (Rolling Window) Metrics

Last4 metrics track performance over the most recent N evaluations (default N=4). These metrics are computed in **CELL 6A** using `add_cumulative_oer_metrics()` with `include_last4_metrics=True`.

`last4_tb_rtr`

- **Type**: Integer (count)
- **Computation**:

    - Count of top blocks from **raters** in the last N evaluations (default N=4)
    - Uses rolling window: finds the last N OERs with `eval_thru_dt <= snpsht_dt` and counts those with `rater_box == 50.0`
    - If fewer than N evaluations exist, counts all available
- **Source**: Module 3 in `add_cum_oer_metrics_mod_working.py`
- **Note**: Window size configurable via `last4_window_size` parameter (default: 4)

`last4_tb_snr`

- **Type**: Integer (count)
- **Computation**:
    - Count of top blocks from **senior raters** in the last N evaluations (default N=4)
    - Uses rolling window: finds the last N OERs with `eval_thru_dt <= snpsht_dt` and counts those with `snr_rater_box == 50.0`
- **Source**: Module 3 in `add_cum_oer_metrics_mod_working.py`

`last4_evals_rtr`

- **Type**: Integer (count)
- **Computation**:
    - Count of evaluations from **raters** in the last N evaluations (default N=4)
    - Uses rolling window: finds the last N OERs with `eval_thru_dt <= snpsht_dt` and `rater_box.notna()`
    - Capped at window size (if 10 evaluations exist but window is 4, returns 4)
- **Source**: Module 3 in `add_cum_oer_metrics_mod_working.py`

`last4_evals_snr`

- **Type**: Integer (count)
- **Computation**:
    - Count of evaluations from **senior raters** in the last N evaluations (default N=4)
    - Uses rolling window: finds the last N OERs with `eval_thru_dt <= snpsht_dt` and `snr_rater_box.notna()`
- **Source**: Module 3 in `add_cum_oer_metrics_mod_working.py`

`last4_tb_ratio_rtr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
    - Ratio of top blocks to evaluations from **raters** in the last N evaluations
    - Formula: `last4_tb_rtr / last4_evals_rtr` (returns `NaN` if denominator is 0)
- **Source**: Module 3 in `add_cum_oer_metrics_mod_working.py`

`last4_tb_ratio_snr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
    - Ratio of top blocks to evaluations from **senior raters** in the last N evaluations

- Formula: `last4_tb_snr / last4_evals_snr` (returns `NaN` if denominator is 0)

- **Source**: Module 3 in `add_cum_oer_metrics_mod_working.py`

Pool Metrics

Pool metrics compare an officer's performance to the average performance of other officers in their rating pool (officers with the same rater or senior rater). These metrics are computed in **CELL 6A** using `add_cumulative_oer_metrics()` with `include_pool_metrics=True`.

Pool Membership Determination

For each snapshot date, pool membership is determined by: 1. Finding the officer's current rater/senior rater as of that snapshot date (OER with `eval_thru_dt <= snpsht_dt`, most recent) 2. Identifying all officers with the same rater/senior rater at that snapshot date 3. Computing statistics for that pool (excluding the officer themselves if `pool_exclude_self=True`)

OER-Based Pool Metrics (18 columns total)

Pool metrics compare an officer's performance to the average performance of other officers in their rating pool (officers with the same rater or senior rater). Pool metrics are computed for the 3 core cumulative OER metrics (cum_tb_rcvd, cum_evals_rcvd, cum_tb_rcvd_ratio), with 6 statistics per metric (for both rater and senior rater pools).

Note: Pool metrics are NOT computed for grade-based metrics (co_grade, field_grade) or last4 metrics. Only the core cumulative metrics have pool statistics.

Pool Means (3 metrics × 2 pools = 6 columns)

- `pool_cum_tb_mean_rtr`: Mean of `cum_tb_rcvd_rtr` in rater pool (excluding self)
- `pool_cum_tb_mean_snr`: Mean of `cum_tb_rcvd_snr` in senior rater pool (excluding self)
- `pool_cum_eval_mean_rtr`: Mean of `cum_evals_rcvd_rtr` in rater pool (excluding self)
- `pool_cum_eval_mean_snr`: Mean of `cum_evals_rcvd_snr` in senior rater pool (excluding self)
- `pool_tb_ratio_mean_rtr`: Mean of `cum_tb_rcvd_ratio_rtr` in rater pool (excluding self)
- `pool_tb_ratio_mean_snr`: Mean of `cum_tb_rcvd_ratio_snr` in senior rater pool (excluding self)

Computation: - For each snapshot date, find all officers in the pool - Compute mean of the metric for pool members (excluding the officer themselves if `pool_exclude_self=True`) - Returns `NaN` if pool size < `pool_min_size` (default: 1)

Pool Differences (3 metrics × 2 pools = 6 columns)

- `cum_tb_diff_rtr`: `cum_tb_rcvd_rtr - pool_cum_tb_mean_rtr`
- `cum_tb_diff_snr`: `cum_tb_rcvd_snr - pool_cum_tb_mean_snr`
- `cum_eval_diff_rtr`: `cum_evals_rcvd_rtr - pool_cum_eval_mean_rtr`
- `cum_eval_diff_snr`: `cum_evals_rcvd_snr - pool_cum_eval_mean_snr`
- `tb_ratio_diff_rtr`: `cum_tb_rcvd_ratio_rtr - pool_tb_ratio_mean_rtr`
- `tb_ratio_diff_snr`: `cum_tb_rcvd_ratio_snr - pool_tb_ratio_mean_snr`

Computation: - Individual officer's metric minus pool mean - Positive values indicate above-average performance - Returns `NaN` if pool mean is `NaN` (pool too small)

Pool Ranks (3 metrics × 2 pools = 6 columns)

- `cum_tb_rank_rtr`: Ordinal rank (1 = highest) of `cum_tb_rcvd_rtr` within rater pool
- `cum_tb_rank_snr`: Ordinal rank (1 = highest) of `cum_tb_rcvd_snr` within senior rater pool
- `cum_eval_rank_rtr`: Ordinal rank (1 = highest) of `cum_evals_rcvd_rtr` within rater pool
- `cum_eval_rank_snr`: Ordinal rank (1 = highest) of `cum_evals_rcvd_snr` within senior rater pool
- `tb_ratio_rank_rtr`: Ordinal rank (1 = highest) of `cum_tb_rcvd_ratio_rtr` within rater pool
- `tb_ratio_rank_snr`: Ordinal rank (1 = highest) of `cum_tb_rcvd_ratio_snr` within senior rater pool

Computation: - Uses `scipy.stats.rankdata` (or manual ranking) with method='ordinal' - Rank 1 = highest value in pool - Ties receive same rank - Returns `NaN` if pool size < `pool_min_size`

Pool Percentile Ranks (Planned)

Purpose: Capture **ordering** within the pool without assuming equal spacing between ranks. This aligns with the Army's TB allocation practice, where leaders rank their current pool and then allocate scarce top blocks under a global profile constraint.

- `cum_tb_rank_pct_rtr`: Percentile rank of `cum_tb_rcvd_rtr` within rater pool (0–1)
- `cum_tb_rank_pct_snr`: Percentile rank of `cum_tb_rcvd_snr` within senior rater pool (0–1)
- `cum_eval_rank_pct_rtr`: Percentile rank of `cum_evals_rcvd_rtr` within rater pool (0–1)
- `cum_eval_rank_pct_snr`: Percentile rank of `cum_evals_rcvd_snr` within senior rater pool (0–1)
- `tb_ratio_rank_pct_rtr`: Percentile rank of `cum_tb_rcvd_ratio_rtr` within rater pool (0–1)
- `tb_ratio_rank_pct_snr`: Percentile rank of `cum_tb_rcvd_ratio_snr` within senior rater pool (0–1)

Computation: - For each snapshot + pool, compute percentile position within the pool (empirical CDF) - Output normalized 0–1 (1 = highest) - Robust to outliers and skewed distributions

Pool Z-Scores (Planned)

Purpose: Capture **magnitude** of standing within the pool (how far above/below the pool mean), not just order. This is useful when the top of the pool is meaningfully "far" above peers, not just slightly higher.

- `cum_tb_zpool_rtr`: Z-score of `cum_tb_rcvd_rtr` within rater pool
- `cum_tb_zpool_snr`: Z-score of `cum_tb_rcvd_snr` within senior rater pool
- `cum_eval_zpool_rtr`: Z-score of `cum_evals_rcvd_rtr` within rater pool
- `cum_eval_zpool_snr`: Z-score of `cum_evals_rcvd_snr` within senior rater pool
- `tb_ratio_zpool_rtr`: Z-score of `cum_tb_rcvd_ratio_rtr` within rater pool
- `tb_ratio_zpool_snr`: Z-score of `cum_tb_rcvd_ratio_snr` within senior rater pool

Computation: - For each snapshot + pool: `(officer_value - pool_mean) / pool_std` - If pool_std = 0, return `NaN` or 0 (configurable)

Pool Normalized Ranks (3 metrics × 2 pools = 6 columns)

- `cum_tb_rank_norm_rtr`: Normalized rank (0-1, 1 = highest) of `cum_tb_rcvd_rtr` within rater pool
- `cum_tb_rank_norm_snr`: Normalized rank (0-1, 1 = highest) of `cum_tb_rcvd_snr` within senior rater pool
- `cum_eval_rank_norm_rtr`: Normalized rank (0-1, 1 = highest) of `cum_evals_rcvd_rtr` within rater pool
- `cum_eval_rank_norm_snr`: Normalized rank (0-1, 1 = highest) of `cum_evals_rcvd_snr` within senior rater pool
- `tb_ratio_rank_norm_rtr`: Normalized rank (0-1, 1 = highest) of `cum_tb_rcvd_ratio_rtr` within rater pool
- `tb_ratio_rank_norm_snr`: Normalized rank (0-1, 1 = highest) of `cum_tb_rcvd_ratio_snr` within senior rater pool

Computation: - Formula: `(pool_size - pool_rank + 1) / pool_size` - Normalized to 0-1 scale where 1 = highest rank - Returns `NaN` if pool size < `pool_min_size`

Pool Size (2 columns)

- `pool_size_rtr`: Number of officers in rater pool (including self)
- `pool_size_snr`: Number of officers in senior rater pool (including self)

Prestige Pool Metrics (14 columns, optional)

If `include_prestige_pool_metrics=True` and prestige columns exist, pool metrics are also computed for prestige metrics:

Prestige Pool Means (3 metrics × 2 pools = 6 columns)

- `pool_prestige_quarters_mean_rtr`: Mean of `cum_prestige_quarters` in rater pool (excluding self)
- `pool_prestige_quarters_mean_snr`: Mean of `cum_prestige_quarters` in senior rater pool (excluding self)
- `pool_total_quarters_mean_rtr`: Mean of `cum_total_quarters` in rater pool (excluding self)
- `pool_total_quarters_mean_snr`: Mean of `cum_total_quarters` in senior rater pool (excluding self)
- `pool_prestige_ratio_mean_rtr`: Mean of `prestige_quarters_ratio` in rater pool (excluding self)
- `pool_prestige_ratio_mean_snr`: Mean of `prestige_quarters_ratio` in senior rater pool (excluding self)

Prestige Pool Differences (3 metrics × 2 pools = 6 columns)

- `prestige_quarters_diff_rtr`: `cum_prestige_quarters - pool_prestige_quarters_mean_rtr`
- `prestige_quarters_diff_snr`: `cum_prestige_quarters - pool_prestige_quarters_mean_snr`
- `total_quarters_diff_rtr`: `cum_total_quarters - pool_total_quarters_mean_rtr`
- `total_quarters_diff_snr`: `cum_total_quarters - pool_total_quarters_mean_snr`
- `prestige_ratio_diff_rtr`: `prestige_quarters_ratio - pool_prestige_ratio_mean_rtr`
- `prestige_ratio_diff_snr`: `prestige_quarters_ratio - pool_prestige_ratio_mean_snr`

Prestige Pool Ranks (3 metrics × 2 pools = 6 columns)

- `prestige_quarters_rank_rtr`: Ordinal rank (1 = highest) within rater pool
- `prestige_quarters_rank_snr`: Ordinal rank (1 = highest) within senior rater pool
- `total_quarters_rank_rtr`: Ordinal rank (1 = highest) within rater pool
- `total_quarters_rank_snr`: Ordinal rank (1 = highest) within senior rater pool
- `prestige_ratio_rank_rtr`: Ordinal rank (1 = highest) within rater pool
- `prestige_ratio_rank_snr`: Ordinal rank (1 = highest) within senior rater pool

Prestige Pool Normalized Ranks (3 metrics × 2 pools = 6 columns)

- `prestige_quarters_rank_norm_rtr`: Normalized rank (0-1, 1 = highest) within rater pool
- `prestige_quarters_rank_norm_snr`: Normalized rank (0-1, 1 = highest) within senior rater pool
- `total_quarters_rank_norm_rtr`: Normalized rank (0-1, 1 = highest) within rater pool
- `total_quarters_rank_norm_snr`: Normalized rank (0-1, 1 = highest) within senior rater pool
- `prestige_ratio_rank_norm_rtr`: Normalized rank (0-1, 1 = highest) within rater pool
- `prestige_ratio_rank_norm_snr`: Normalized rank (0-1, 1 = highest) within senior rater pool

Source: Module 5 in `add_cum_oer_metrics_mod_working.py`

Ratings Given Metrics

Ratings given metrics track an officer's performance as a rater or senior rater (how many top blocks they gave to others). These metrics are computed in **CELL 6A** using `add_cumulative_oer_metrics()` with `include_ratings_given=True`.

`cum_tb_given_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of top blocks given by this officer as a **rater** up to each snapshot date
  - Uses backward temporal lookup: finds all OERs where `rater == pid_pde` and `eval_thru_dt <= snpsht_dt` and `rater_box == 50.0`
- **Source**: Module 4 in `add_cum_oer_metrics_mod_working.py`

`cum_tb_given_snr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of top blocks given by this officer as a **senior rater** up to each snapshot date
  - Uses backward temporal lookup: finds all OERs where `snr_rater == pid_pde` and `eval_thru_dt <= snpsht_dt` and `snr_rater_box == 50.0`
- **Source**: Module 4 in `add_cum_oer_metrics_mod_working.py`

`cum_evals_given_rtr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of evaluations given by this officer as a **rater** up to each snapshot date
  - Counts any OER where `rater == pid_pde` and `eval_thru_dt <= snpsht_dt` and `rater_box.notna()`
- **Source**: Module 4 in `add_cum_oer_metrics_mod_working.py`

`cum_evals_given_snr`

- **Type**: Integer (cumulative count)
- **Computation**:
  - Cumulative count of evaluations given by this officer as a **senior rater** up to each snapshot date
  - Counts any OER where `snr_rater == pid_pde` and `eval_thru_dt <= snpsht_dt` and `snr_rater_box.notna()`
- **Source**: Module 4 in `add_cum_oer_metrics_mod_working.py`

`cum_tb_given_ratio_rtr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:

    - Ratio of top blocks to evaluations given as a **rater**
    - Formula: `cum_tb_given_rtr / cum_evals_given_rtr` (returns `NaN` if denominator is 0)
- **Source**: Module 4 in `add_cum_oer_metrics_mod_working.py`

`cum_tb_given_ratio_snr`

- **Type**: Float (0.0 to 1.0)
- **Computation**:
    - Ratio of top blocks to evaluations given as a **senior rater**
    - Formula: `cum_tb_given_snr / cum_evals_given_snr` (returns `NaN` if denominator is 0)
- **Source**: Module 4 in `add_cum_oer_metrics_mod_working.py`

Summary

Total Column Count by Category

- **Prestige Metrics**: 3-7 columns (depending on optional modules)
- **OER Cumulative Metrics**: 6 columns
- **Grade-Based Metrics**: 12 columns
- **Last4 Metrics**: 6 columns
- **Pool Metrics**: 18 columns (OER-based) + 14 columns (prestige-based, optional)
- **Ratings Given Metrics**: 6 columns

Key Computation Principles

1. **Temporal Integrity**: All metrics respect snapshot dates—only data available "as of" each date is used
2. **Deduplication**: Each OER is counted exactly once, even if it appears multiple times in merged data
3. **Backward Lookup**: For each snapshot date, metrics are computed using all OERs with `eval_thru_dt <= snpsht_dt`
4. **Vectorization**: Operations are performed on entire arrays/columns, not row-by-row
5. **Pool Exclusion**: Pool metrics exclude the officer themselves when computing pool means (if `pool_exclude_self=True`)

References

- Main computation function: `add_cumulative_oer_metrics()` in `add_cum_oer_metrics_mod_working.py`
- Detailed documentation: `ADD_CUM_OER_METRICS_OVERVIEW.md`
- Pipeline overview: `520_PIPELINE_COX_OVERVIEW.md`